Exercise 4: Clustering

Starting with this exercise, you will work with 3D pointclouds as input data recorded with a depth camera instead of 2D images. In this exercise, the pointclouds should be clustered, such that in the ideal case all points of an object are associated to the same cluster. A good clustering result is crucial for a reliable object recognition, which you will implement as the final challenge of the course. An example of a good clustering result can be seen in Figure 2. You will only need to process the spatial information of the pointcloud, color information is not used yet. An example pointcloud of the dataset is shown in Figure 1.

We will use the library Open3D to process the pointclouds. You can find a tutorial on the official site. However, we will mostly use this library for its I/O capabilities and for visualization. For implementing the Clustering algorithm, we will transform the points to NumPy arrays and perform the calculations on them.

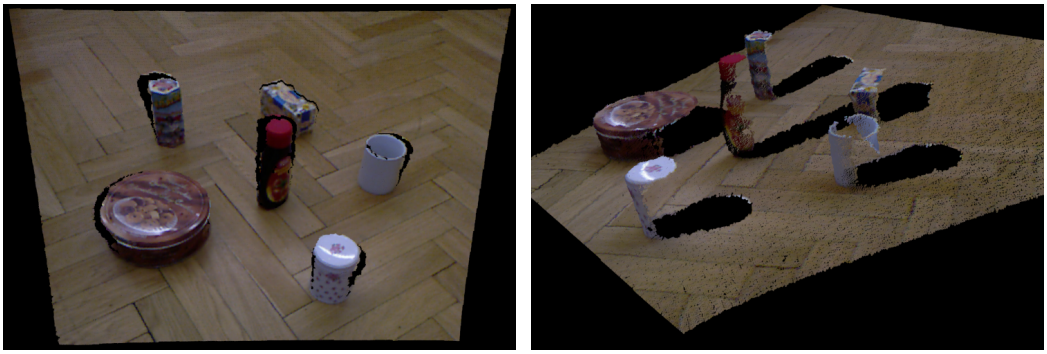


Figure 1: Example pointcloud from the dataset shown from two different viewpoints. Note the difficulties capturing the cup as one object.

Part 1: Implementation (12 points)

In this exercise, you will implement and compare different Clustering algorithms to find the best for our input data. In Figure 2, you see a successful clustering result.

In Figure 3 you can see the result of the RANSAC algorithm with the detected plane colored in red.

0. Plane Detection (2 Bonus points)

Before we can start with the clustering algorithm, we need to remove the ground plane from the pointcloud. To do so, Open3D provides the function segment_plane with a short tutorial here. You can see a successfully detected plane in Figure 3.



Figure 2: Detected clusters marked with different colors of the down-sampled point cloud.

However, we want to encourage you to implement many parts of the final object detection pipeline yourself, therefore you can earn 2 bonus points if you adapt your RANSAC algorithm from exercise 2 to detect the plane.

To reduce computation time, the pointclouds are down-sampled before we process them further. Open3D provides two different down-sampling approaches which you can choose from: `uniform_down_sample` and `voxel_down_sample`. If you want to use the built-in plane segmentation, you can skip to the next section.

The basic steps of RANSAC are the same as in exercise 2. However, this time we will only sample 3 points and from those derive the plane equation $\mathbf{nx} = \mathbf{np}$ where $\mathbf{n} \in \mathbb{R}^3$ is the normal vector of the plane and $\mathbf{p} \in \mathbb{R}^3$ is a point of the plane. This can be written as $ax + by + cz + d = 0$ which is also the form returned by the Open3D function. A point is considered an inlier, if its distance to the proposed plane is smaller than a certain threshold we provide as a parameter.

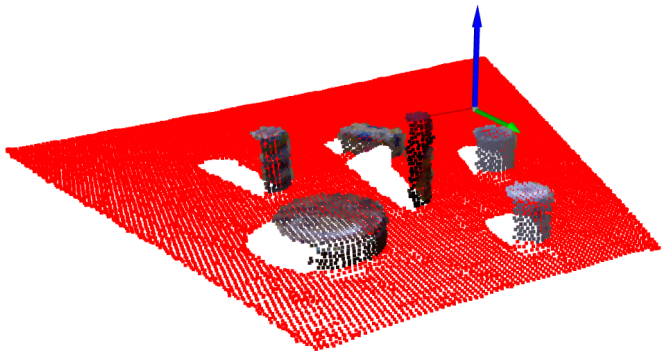


Figure 3: Successfully detected plane of the down-sampled point cloud with inliers colored in red.

After RANSAC has divided the set of points in inliers and outliers, the plane fit can be further improved by finally considering all inlier points to calculate the plane parameters a, b, c and d ($ax + by + cz + d = 0$). For that, an overdetermined system of equations has to be solved, which can be done with the least squares alogorithm, as follows:

$$\begin{pmatrix} x_1 & y_1 & z_1 & 1 \\ x_2 & y_2 & z_2 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & z_n & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Parameter d is free to choose, such that for e.g. $d = -1$ we get the following simplified system of equations:

$$\begin{pmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

This system of equations is overdetermined (n equations for 3 unknowns, n = number of inliers). To get the least-squares solution of this system, NumPy offers the function `np.linalg.lstsq`.

A plane can be defined such that points on one side of it are in a "positive" space, and others in a "negative" space, depending on the direction of the normal. For our purposes this does not matter. Ensure that the normal vector (i.e., coefficients of the plane equation) has a positive z-component, and make sure the rest of the vector components are kept consistent with this. You should update the inliers after refining the plane.

Implement the `fit_plane` in **fit_plane.py**.

1. *K*-Means Clustering (3 points)

After removing the ground plane, we can start with the clustering. The first clustering algorithm you need to implement is the k-means algorithm. The algorithm was described in the lecture, but we will also outline the basic steps here:

1. Randomly select k start cluster centers $\boldsymbol{c}_k$
2. Assign each point $\boldsymbol{x}_i \in X$ to the nearest cluster center (squared Euclidean distance).
3. Recalculate the cluster centers as means of the assigned points.

$$\boldsymbol{c}'_k = \frac{1}{|C_k|} \sum_{\boldsymbol{x}_j \in C_k} \mathbf{x}_j$$

where $|C_k|$ is the number of points in the cluster C_k.

4. Loop step 2 and 3 until the cluster assignments no longer change or a maximum number of iterations is reached defined by the paramter `max_singlerun_iterations`.

To initialize the cluster centers, we will use the Forgy method that simple chooses k random points from the input data. You do not need to calculate the distance of the cluster centers to all points manually. You can use a function from the scipy.spatial.distance package (e.g. `scipy.spatial.distance.cdist`).

The goal of the k-means algorithm is to minimize the so called inertia which is defined as

$$\sum_{i=0}^{n} \min_{\boldsymbol{c}_j \in C} \left(||\mathbf{x}_i - \boldsymbol{c}_j||^2 \right).$$

Since the k-means is very dependent on the chosen starting cluster centers and relatively cheap to run, we will run the algorithm multiple times, defined by the `n_iterations` parameter. Your final clusters should be the ones that produce the smallest inertia.

Note: For testing and debugging of all clustering algorithms, we recommend to use the pointcloud stored in `image000.pcd` since the clusters are well separated. Some of the provided pointclouds are tricky for most clustering algorithms which makes it difficult to find bugs in your code.

Note: If you have some form of color vision deficiencies and have problems seeing the colors of the clusters, you can choose a different colormap for the function `plot_clustering_results` with the parameter `cmap`. The colormaps `"viridis"` or `"magma"` might be good options.

Implement the `kmeans` function in **clustering.py**.

2. Silhouette Score (2 points)

The k-means algorithm requires us to specify the number of clusters beforehand. This is not ideal if you don't know the data beforehand. There are multiple methods how to find the best number of clusters k. You will first implement the so called silhouette value proposed by Peter Rousseeuw [1]. It is a score between -1 and 1 specifying how similar a sample is to its own cluster compared to other clusters. The higher the value, the better the sample fits to its current cluster.

For each datapoint $\mathbf{x}_i$, it is calculated using the mean intra-cluster distance a_i and the mean nearest-cluster distance b_i. The mean intra-cluster distance is defined as

$$a_i = \frac{1}{|C_I| - 1} \sum_{j \in C_I, i \neq j} d_{ij}$$

where $|C_I|$ is the number of points in the same cluster I as the datapoint and d_{ij} is the (Euclidean) distance between $\mathbf{x}_i$ and $\mathbf{x}_j$. The mean nearest-cluster distance is defined as

$$b_i = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} d_{ij}$$

therefore it is the mean distance of $\mathbf{x}_i$ to all points of the next closest cluster C_J. With these values we can calculate the silhoutte score for each point $\mathbf{x}_i$ as

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

When a cluster $C_K = \{\mathbf{x}_k\}$ contains only a single datapoint, a_k is not defined and we define $s_k = 0$. The final silhouette score should be the mean over all s_i for the whole dataset.

Implement the function `silhouette_score` in **helper_functions.py**.

3. Iterative k-means (1 point)

The iterative k-means function should now loop over multiple number of clusters, execute for each the k-means algorithm, calculate the mean silhouette score and return the clusters with the highest mean silhouette score.

> **Note:** If you had problems implementing the previous steps, you can replace your implementation of k-means or the silhouette score with existing solutions. See the "Forbidden Commands" section for references.

Implement the function `iterative_kmeans.py` in **clustering.py**.

4. G-Means Clustering (3 points)

The silhouette scores are a good measure to determine which number of clusters is the best after they have been calculated for an arbitrary number of clusters. However, this number of maximum clusters still has to be defined. It would be preferable, if our clustering algorithm stops when the number of clusters fits the data sufficiently. An approach to do so is the Gaussian-means or G-means algorithm proposed by Hamerly and Elkan in 2003 [2]. The idea is, you start with a low number of k-means clusters (e.g., $k = 1$). At each iteration, a cluster is split into two if its data does not seem to come from a Gaussian distribution. The test is based on the Anderson-Darling test for one-dimensional data.

The steps of the algorithm are as follows:

1. Start with k=1 cluster. The first center is the mean of all points.
2. Run k-means with all centers and all points X to get new centers and labels.
3. For each cluster $C_I \subseteq X$, initialize two new centers ("children") with the formula

$$\mathbf{c}'_{i_{1,2}} = \mathbf{c}_i \pm \mathbf{s}_i \sqrt{2\lambda_i/\pi}$$

where $\mathbf{s}_i$ is the main principal component of the points of the cluster. This is the eigenvector with the highest eigenvalue λ_i of the covariance matrix $\mathrm{Cov}(C_I) \in \mathbb{R}^{3\times3}$ of the points of the cluster. This principle is shown in Figure 4. You can use the functions provided by NumPy to get the covariance matrix and the eigenvectors.

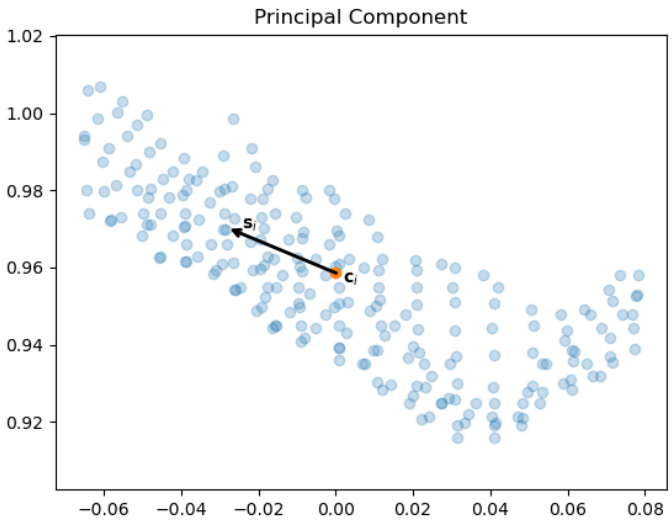


Figure 4: First principal component $\boldsymbol{s}_i$ of a cluster projected to 2D. The vector is drawn from the center of the cluster $\boldsymbol{c}_i$.

4. For each cluster, run k-means on these two centers and only the subset of points $\boldsymbol{x}_i \in C_I$ to retrieve the two updated centers $\mathbf{c}'_{i_{1,2}}$.

5. For each initial cluster, project all points $\boldsymbol{x}_i \in C_I$ of the initial cluster onto the vector $\mathbf{v} = \mathbf{c}'_{i_1} - \mathbf{c}'_{i_2}$ by calculating

$$\mathbf{x}'_i = \frac{\langle \mathbf{x}_i, \mathbf{v} \rangle}{||\mathbf{v}||^2}.$$

6. For each cluster C_I, perform the Anderson-Darling normality test using the function `anderson()` of the package `scipy.stats`. It returns the test statistic and critical values when to reject the hypothesis of the data being normal distributed. Since our data is not *exactly* normally distributed, we pick the highest critical value and increase it further with the `tolerance` parameter. You can use this code snippet for the check if the points of a cluster fulfill the condition.

```
# Perform the Anderson-Darling test for normality with the projected
    points
estimation, critical, _ = anderson(x_projected)
if estimation <= critical[-1] * tolerance:
    is_gaussian = True
```

7. If the data fulfils the condition to be normally distributed, keep $\mathbf{c}_i$. Otherwise replace it with the two new centers $\mathbf{c}'_{i_{1,2}}$.

8. Repeat from step 2 until no more centers are added.

For more details, you can check the paper in [2].

Implement the function `gmeans` in **clustering.py**.

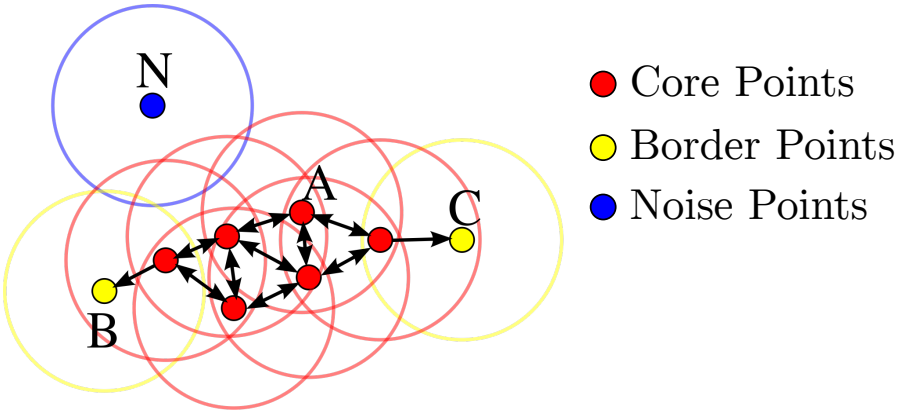


Figure 5: Visualization of the different types of points in DBSCAN. Adapted from Chire - Own work, CC BY-SA 3.0.

5. DBSCAN (3 points)

The k-means algorithms and its extensions are very efficient and work well for many cases. However, if the data does not have a uniform, spherical shape or if there is a lot of noise, k-means does not perform great. The algorithm *"density-based spatial clustering of applications with noise"* (DBSCAN) attempts to solve some of these shortcomings. It was introduced by Ester et al. in 1996 [3] as a density-based non-parametric clustering algorithm. The key idea is that for each point of a cluster the neighborhood of a given radius has to contain at least a minimum number of points, i.e. the density in the neighborhood has to exceed some threshold. Points that fulfil this criterion are considered as core points. Points on the border of the cluster are considered part of the cluster if they are in an ε-neighborhood of a core point. All other points are considered to be noise. An illustration of the different point types can be seen in Figure 5.

For the basic algorithm refer to the lecture slides. You should implement the algorithm described by the steps here:

1. Select a point x_i from the dataset which has not been visited (i.e., does not have a label yet).
2. Retrieve all points in an ε-neighborhood (i.e., they are density-reachable)
3. If the number of points is smaller than the parameter `min_samples`, mark the point as noise (i.e., label=-1) and start again at step 1.
4. If the number of points in the neighborhood is bigger than `min_samples`, $\boldsymbol{x}_i$ is a core point.
5. Check all neighboring points if they are core points and repeat this step to iteratively expand the cluster. Additionally, all neighboring points are labeled as part of the cluster

6. If there are no more new core points, we can label all remaining neighboring points (i.e. border points) with the cluster label and start again at step 1 until all points have been processed.

Be careful to not expand the cluster, if a point is only reachable via a border point and is therefore not density-reachable. Additionally, be aware that a point that has initially been labeled as noise could be a border point of a cluster which is processed later.

Implement the function `dbscan` in **clustering.py**.

Useful commands

- `np.cross` - Return the cross product of two (arrays of) vectors
- `np.dot` - Return the dot product of two arrays. If both arrays are 1-D arrays, it is the inner product of vectors.
- `np.expand_dims` Insert a new axis that will appear at the axis position in the expanded array shape.
- `np.linalg.eig` Compute the eigenvalues and right eigenvectors of a square array.
- `np.cov` Estimate the covariance matrix.
- `np.mean` Compute the arithmetic mean. By providing the optional `axis` parameter, you can specify along which axis to calculate the mean.
- `np.asarray(pcd.points)` - Get the 3D coordinates of all points of a pointcloud as a NumPy Array.
- `o3d.geometry.PointCloud.select_by_index` Function to select points from input pointcloud into output pointcloud
- `o3d.utility.Vector3dVector` Convert float64 NumPy array of shape (n, 3) to Open3D format.
- `scipy.spatial.distance.cdist` Compute distance between each pair of the two collections of inputs. For other functions that calculate distances between sets of points you can refer to the whole `scipy.spatial.distance` package.
- `scipy.stats.anderson` Anderson-Darling test for data coming from a particular distribution.

Forbidden Commands

All clustering algorithms of sklearn.cluster and `o3d.geometry.PointCloud.cluster_dbscan` should not be used. However, they might be useful to compare them to your clustering algorithms or to use them for other sections of the exercise, if your implementation does not work well. The function `sklearn.metrics.silhouette_score` should not be used, but can be used for iterative k-means, if your implementation of the silhouette score is not working properly.

Part 2: Documentation (6 points)

- Answer each numbered question separately, with the answer labelled according to the experiment and question number. For example, Question 2 should be numbered with "2". You do not need to include the question text.
- Include images to support your answers. These should be referenced in the answers. The document should include information about what parameter settings were used to generate images so that they can be reproduced. Feel free to use your own images but if you do so, upload the original image as well so we can reproduce your results.

  > **Note:** When you are in the Open3D visualization window, you can use the `P`-key to take a screen capture. Other commands can be listed with the `H`-key.

- Your answers should attempt an explanation of why an effect occurs, not just describe what you see in the supporting images.
- Be precise. "It is better" or similarly vague statements without any additional information are not sufficient.

Points may be deducted if your report does not satisfy the above points.

1. Compare the clustering algorithms you have implemented. You should compare them for different pointclouds. Pick pointclouds, where you see differences between the different algorithms and point them out in the report. What are advantages and disadvantages of the different approaches? (3 points)
2. Plot the runtimes of the different clustering algorithms on at least two clouds, with at least two different levels of downsampling. How do the methods compare? (2 points)
3. How do the parameters of DBSCAN `eps` and `min_sample_points` influence the clustering results? (1 point)

You must specify the parameters used in all of your screenshots.

> **Note:** If some of your algorithms don't work well or you did not implement them, you can use the implementation from scikit-learn for the documentation. Since G-means is not implemented in this library, you can pick any other algorithm from the library instead.

Maximum 1500 words, arbitrary number of pages for result images (referenced in the text). Violating the text limit will lead to point deductions!

Honor Code

Add the honor code to the beginning of the documentation:

For the assignment I followed the honor code:

- Rule 1: Don't look at solutions or code of other students; everything you submit should be your own work
- Rule 2: Don't share your solution code with others; however, discussing ideas or general strategies is fine and encouraged
- Rule 3: If you use code from somewhere you need to cite the source directly in your code (e.g., StackOverflow). If the source is a generative AI tool (e.g., ChatGPT) provide a link to the specific Chat.

I know that I have full responsibility for what I submit. I made sure the assignment allows the teaching staff to evaluate which competences I have acquired. I am aware that plagiarism is prohibited and may lead to failure of this course.

Assistance

Please keep to the following hierarchy whenever there are questions or problems:

1. Forum: Use the TUWEL forum to discuss your problems.
2. Email for questions or in-person help requests: machinevision@acin.tuwien.ac.at

Upload

Please upload all Python files, as well as your documentation (pdf-format) **as one ZIP-file** via TUWEL.

References

[1] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," en, *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, Nov. 1987. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/0377042787901257.

[2] G. Hamerly and C. Elkan, "Learning the k in k-means," *Advances in neural information processing systems*, vol. 16, 2003. [Online]. Available: https://proceedings.neurips.cc/paper/2003/file/234833147b97bb6aed53a8f4f1c7a7d8-Paper.pdf.

[3] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise.," in *kdd*, Issue: 34, vol. 96, 1996, pp. 226–231. [Online]. Available: https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf.